# Architecture CNN avancée

Computer Vision - Séance 2

# 1. Rappel : Fonctionnement des CNN

**Problème des réseaux fully-connected** :

- Image 224x224x3 = 150,528 pixels
- Première couche FC de 1000 neurones = 150 millions de paramètres
- Pas d'exploitation de la structure spatiale de l'image

**Intuition des CNN** :

- Les motifs visuels sont **locaux** (bords, textures, formes)
- Les motifs sont **invariants par translation** (un chat reste un chat peu importe sa position)
- Hiérarchie de complexité : pixels -> bords -> textures -> parties -> objets

# 1.1. Convolution

**Principe** : appliquer un filtre (kernel) qui "glisse" sur l'image

**Image d'entrée** **Filtre 3x3** **Sortie**

$$\begin{matrix} 1 & 2 & 3 & 0 \\ 0 & 1 & 2 & 1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \end{matrix} * \begin{matrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{matrix} = \begin{matrix} 7 & 6 \\ 4 & 5 \end{matrix}$$

Position (0,0) : 1*1 + 2*0 + 3*1 + 0*0 + 1*1 + 2*0 + 1*1 + 0*0 + 1*1 = 7

Position (0,0) : 1*1 + 2*0 + 3*1 + 0*0 + 1*1 + 2*0 + 1*1 + 0*0 + 1*1 = 7

Position (1,0) : 2*1 + 3*0 + 0*1 + 1*0 + 2*1 + 1*0 + 0*1 + 1*0 + 2*1 = 6

Position (0,1) : 0*1 + 1*0 + 2*1 + 1*0 + 0*1 + 1*0 + 2*1 + 1*0 + 0*1 = 4

Position (1,1) : 1*1 + 2*0 + 1*1 + 0*0 + 1*1 + 2*0 + 1*1 + 0*0 + 1*1 = 5

# 1.2. Fonction d'activation (ReLU)

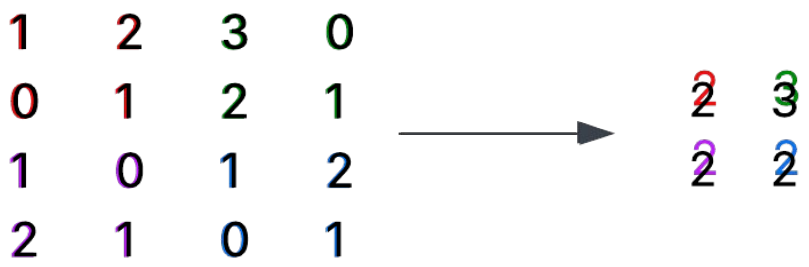
**Formule** : ReLU(x) = max(0, x)

**Pourquoi ReLU ?**

$$\begin{matrix} -3 & 6 \\ -5 & 7 \end{matrix} \longrightarrow \begin{matrix} 0 & 6 \\ 0 & 7 \end{matrix}$$

- Introduit la **non-linéarité** (sinon empilement de convolutions = une seule convolution)
- Calcul rapide (simple comparaison)
- Atténue le problème du vanishing gradient (gradient = 1 si x > 0)

# 1.3. Pooling (sous-échantillonnage)

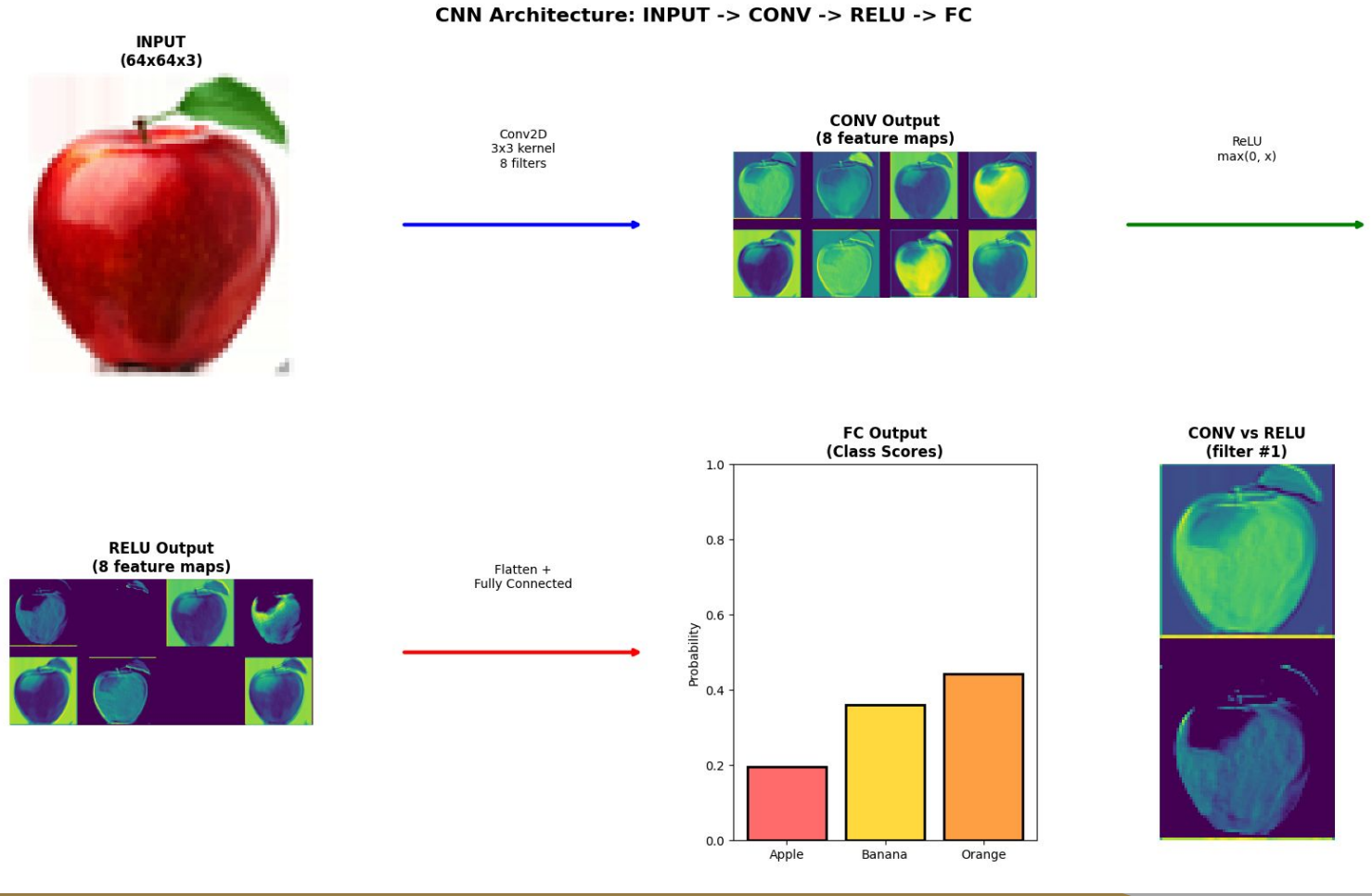



**Objectifs** :

- Réduction dimensionnelle : divise par 2 la taille spatiale
- Invariance locale : petit déplacement du motif = même sortie
- Réduction du sur-apprentissage : moins de paramètres

# 1.4 Architecture type d'un CNN

- `INPUT -> FC` implémente un classifieur linéaire
- `INPUT -> CONV -> RELU -> FC`
- `INPUT -> [CONV -> RELU -> POOL] * 2 -> FC -> RELU -> FC` Ici, il y a une couche de CONV unique entre chaque couche POOL
- `INPUT -> [CONV -> RELU -> CONV -> RELU -> POOL] * 3 -> [FC -> RELU] * 2 -> FC` Ici, il y a deux couches CONV empilées avant chaque couche POOL.


CNN Architecture: INPUT -> CONV -> RELU -> FC
INPUT
(64x64x3)
Conv2D
3x3 kernel
8 filters
CONV Output
(8 feature maps)
ReLU
max(0, x)
RELU Output
(8 feature maps)
Flatten +
Fully Connected
FC Output
(Class Scores)
Probability
1.0
0.8
0.6
0.4
0.2
0.0
Apple
Banana
Orange
CONV vs RELU
(filter #1)

# 2. Evolution historique

- **1998** : LeNet-5 (LeCun) - 5 couches, reconnaissance de chiffres
- **2012** : AlexNet - 8 couches, victoire ImageNet (top-5 error : 15.3%)
- **2014** : VGGNet - 16-19 couches
- **2015** : ResNet - jusqu'à 152 couches
- **2017** : MobileNet - optimisation pour l'embarqué

# 2.1 LeNet-5 (LeCun, 1998)

**Principe** : Premier CNN moderne conçu pour la reconnaissance de chiffres manuscrits (MNIST). Introduit les concepts fondamentaux : convolutions locales, partage de poids et sous-échantillonnage spatial.

**Architecture** :

- Input : 32x32x1 (image en niveaux de gris)
- C1 : Conv 5x5, 6 filtres -> 28x28x6
- S2 : Average Pooling 2x2 -> 14x14x6
- C3 : Conv 5x5, 16 filtres -> 10x10x16
- S4 : Average Pooling 2x2 -> 5x5x16
- C5 : Conv 5x5, 120 filtres -> 1x1x120
- F6 : FC 84 neurones
- Output : FC 10 classes (chiffres 0-9)

Total : ~60,000 paramètres

# 2.2. AlexNet

**Principe** :

- Architecture séquentielle simple : couches empilées les unes après les autres
- Schéma typique : [Conv -> ReLU -> Pool] x N -> Flatten -> [FC -> ReLU] x M -> Softmax
- Augmentation progressive des canaux (ex: 96 -> 256 -> 384 -> 256) pour capturer des features de plus en plus abstraites
- Réduction progressive de la résolution spatiale via pooling (227 -> 55 -> 27 -> 13 -> 6)
- Couches FC massives en fin de réseau (4096 -> 4096 -> 1000) concentrant la majorité des paramètres

**Coût computationnel :**

- Nombre de paramètres croît avec la profondeur
- Mémoire GPU limitée
- Temps d'entraînement prohibitif

# 2.3. VGG - Simplicité et profondeur (2014, Simonyan & Zisserman)

**Principe architectural :**

- Convolutions uniformes 3x3 :
  - Stride = 1, padding = 1 (conserve la dimension spatiale)
  - Deux conv 3x3 = champ réceptif équivalent à une conv 5x5
  - Trois conv 3x3 = champ réceptif équivalent à une conv 7x7
  - Avantage : plus de non-linéarités (ReLU) + moins de paramètres
- Structure en blocs :
  - Conv3x3 -> ReLU -> Conv3x3 -> ReLU -> MaxPool 2x2
  - Doublement des filtres après chaque pooling (64 -> 128 -> 256 -> 512)

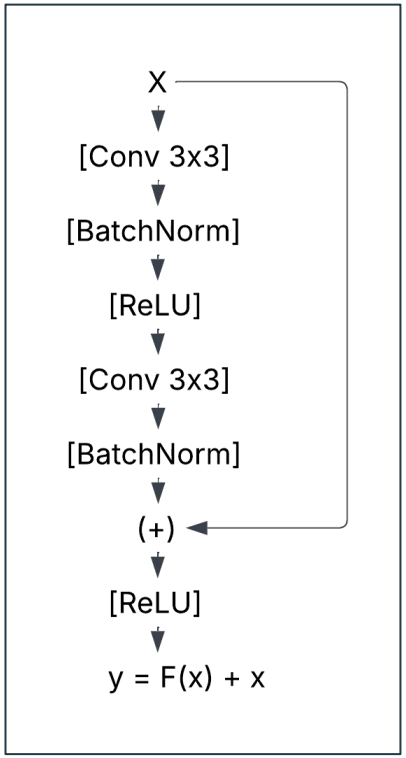
# 2.4. ResNet - Connexions résiduelles (2015, He et al.)

Innovation clé : **le bloc résiduel**

- Idée : au lieu d'apprendre H(x), apprendre F(x) = H(x) - x
- Sortie : y = F(x) + x (connexion de saut / skip connection)

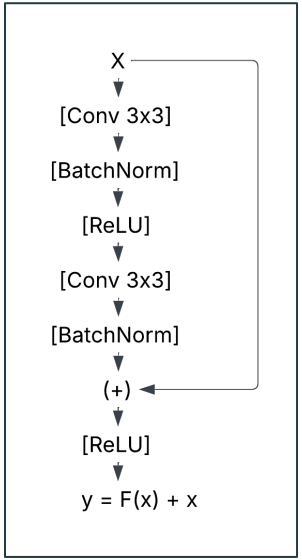
**Pourquoi ca fonctionne ?**

- Gradient direct : le gradient peut "sauter" les couches via la connexion identite
- Optimisation facilitée : apprendre F(x) = 0 est plus facile qu'apprendre H(x) = x
- Pas de paramètres supplémentaires : l'addition est gratuite

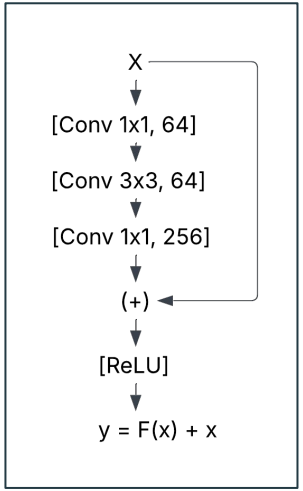


# 2.4. ResNet - Type de block

| Type | Utilisé dans | Structure |
|---|---|---|
| **Basic Block** | ResNet-18/34 | Conv3x3 -> BN -> ReLU -> Conv3x3 -> BN |
| **Bottleneck** | ResNet-50/101/152 | Conv1x1 -> Conv3x3 -> Conv1x1 (réduction de dimension) |




Basic Block




Bottleneck

# 2.5. MobileNet – Efficacité et embarqué

**Objectif** : réduire drastiquement le coût de calcul tout en préservant la précision

**Innovation clé** : Convolutions séparables en profondeur

Convolution standard :

- Filtre $K \times K$ applique sur TOUS les canaux simultanément
- Coût : $K \times K \times C_{in} \times C_{out}$ opérations par pixel

Convolution separable = Depthwise + Pointwise

# 2.5. MobileNet – Depthwise + Pointwise

**Depthwise** (convolution en profondeur)

Applique un filtre $K \times K$ par canal d'entrée, indépendamment.

- Chaque canal est traité séparément
- Pas de mélange d'information entre canaux
- Coût : $K \times K \times C_{in}$ opérations par pixel

**Pointwise** (convolution 1x1)

Convolution avec des filtres de taille 1x1 pour combiner les canaux.

- Mélange l'information entre canaux
- Change le nombre de canaux ($C_{in} \rightarrow C_{out}$)
- Cout : $C_{in} \times C_{out}$ opérations par pixel

# 2.6. Récapitulatif

| Critère | LeNet-5 | AlexNet | VGG-16 | ResNet-50 | MobileNetV2 |
|---|---|---|---|---|---|
| **Paramètres** | 60K | 62M | 138M | 25.6M | 3.4M |
| **Taille modèle** | ~0.2 MB | ~240 MB | ~530 MB | ~100 MB | ~14 MB |
| **GFLOPs** | 0.0004 | 0.7 | 15.5 | 3.8 | 0.3 |
| **Top-1 ImageNet** | N/A (MNIST) | 63.3% | 71.5% | 76.0% | 72.0% |
| **Latence GPU** | N/A | ~200 img/s | ~20 img/s | ~100 img/s | ~300 img/s |
| **Latence CPU mobile** | Temps reel | Lent | Inutilisable | ~7 img/s | ~30 img/s |
| **Transfer learning** | Non | Moyen | Excellent | Excellent | Bon |
| **Déploiement edge** | Oui | Non | Non | Difficile | Oui |

# 3. Transfer Learning

# 3.1 Problématique de l'entraînement

Entrainement from scratch nécessite :

- **Enorme dataset** : ImageNet = 1.2M images, 1000 classes
- **Ressources GPU** : ResNet-50 ~ 14 jours sur 1 GPU, quelques heures sur 8 GPUs
- **Expertise** : choix des hyperparamètres (learning rate, batch size, augmentation)

**Pourquoi le Transfer Learning ?**

Les premières couches d'un CNN apprennent des **features génériques** :

- Couches 1-2 : edges, couleurs, textures simples
- Couches intermédiaires : motifs, formes géométriques
- Couches profondes : features spécifiques au dataset (visages, voitures, etc.)

# 3.2 Modèles pré-entraînés

**Définition :**

Un modèle **pré-entraîné** est un réseau dont les poids ont déjà été optimisés sur un large dataset (généralement ImageNet).

| Framework | Commande | Modèles disponibles |
|---|---|---|
| PyTorch | `torchvision.models` | ResNet, VGG, MobileNet, EfficientNet, ViT... |
| TensorFlow/Keras | `tf.keras.applications` | Mêmes modèles + NASNet, Xception... |
| Hugging Face | `timm` library | 700+ modèles pré-entraînés |

# Done

# 3.3 Stratégies de Transfer Learning

**Stratégie 1 :** Feature Extraction (extracteur de features)

On **gèle** toutes les couches convolutives et on remplace uniquement la dernière couche (classificateur).

**Quand l'utiliser** :

- - Petit dataset (<1000 images)
- - Dataset similaire à ImageNet (objets, animaux, scènes)
- - Ressources limitées

# 3.3 Stratégies de Transfer Learning

**Stratégie 2 :** Fine-tuning (ajustement fin)

On **dégèle** certaines couches (généralement les dernières) et on les ré-entraîne avec un faible learning rate.

**Quand l'utiliser** :

- Dataset de taille moyenne (1000-10000 images)

- Dataset différent d'ImageNet (images médicales, satellite, etc.)

# 3.3 Stratégies de Transfer Learning

**Stratégie 3 :** Fine-tuning progressif (Gradual Unfreezing)

Dégeler les couches **progressivement**, des plus profondes aux plus basses.

1. Entraîner le classificateur (1-2 epochs)
2. Dégeler la dernière couche convolutive (2-3 epochs)
3. Dégeler la couche précédente (2-3 epochs)
4. Continuer jusqu'à convergence

**Avantage** : évite de "casser" les features génériques des premières couches.

# 3.4. Bonnes pratiques

## Learning rate

| Stratégie | Learning rate recommandé |
|---|---|
| Feature extraction | 1e-3 à 1e-2 (classifieur seul) |
| Fine-tuning | 1e-5 à 1e-4 (10-100x plus faible) |
| Couches basses | 1e-6 (si dégelées) |

## Data augmentation

Essentielle pour éviter l'overfitting sur petits datasets :

- **Géométrique** : rotation, flip, crop, scale
- **Photométrique** : brightness, contrast, saturation
- **Avancée** : Mixup, CutMix, RandAugment

## Normalisation

**Toujours** utiliser les mêmes statistiques de normalisation que le modèle pré-entraîné (moyennes et écarts-types ImageNet).

# 4.1. métriques rappel

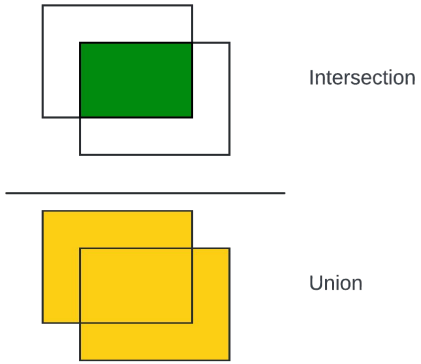
**IoU** (Intersection over Union) : mesure le chevauchement entre la boite prédite et la vérité terrain

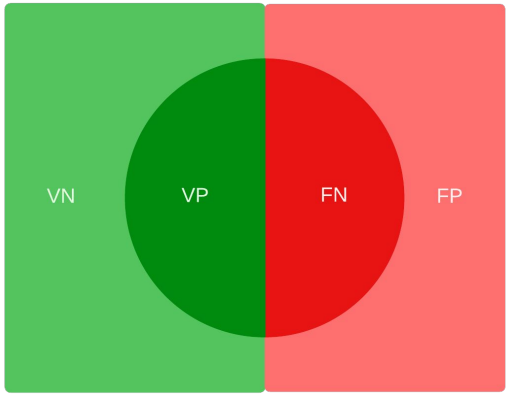
- IoU = (aire intersection) / (aire union)
- Seuil typique : IoU >= 0.5 pour considérer une détection correcte

**Précision** : parmi les détections, combien sont correctes ? (TP / (TP + FP))

**Recall** : parmi les objets réels, combien sont détectés ? (TP / (TP + FN))

**F1-score** : Combinaison de précision et rappel (TP / TP + ½(FP+FN))



# 4.2. Précision vs Coût

Métriques de précision

- **Top-1 Accuracy** : prédiction correcte = classe la plus probable
- **Top-5 Accuracy** : classe correcte dans les 5 premières prédictions
- **mAP** (detection) : mean Average Precision

Impact du Transfer Learning

- Pre-entraîné ImageNet: +10-20% de précision sur petits datasets
- Fine-tuning : adapter les dernières couches au domaine cible
- Tous les modèles disponibles pre-entraînés sur PyTorch/TensorFlow

# 4.3 Temps d'inférence

**Contraintes selon l'application**

| Application | Latence max | FPS minimum |
|---|---|---|
| Vidéo temps réel | 33 ms | 30 FPS |
| Robotique | 100 ms | 10 FPS |
| Batch processing | - | - |
| Application mobile | 100 ms | 10 FPS |

**Facteurs influents**

Matériel :

- GPU serveur (V100, A100) : 100-1000x plus rapide que CPU
- GPU embarqué (Jetson) : 10-50x plus rapide que CPU mobile
- NPU/TPU mobile : optimisé pour les CNN

Optimisations :

- Quantification INT8 : 2-4x accélération
- Pruning : réduction des poids inutiles
- TensorRT / CoreML : compilation optimisée

# 4.4 Taille du modèle et déploiement

| Plateforme | RAM disponible | Taille max modèle |
|---|---|---|
| Serveur | 16-256 GB | Illimitée |
| Desktop | 8-32 GB | ~1 GB |
| Mobile | 2-8 GB | ~100 MB |
| IoT/Edge | 512 MB - 2 GB | ~50 MB |
| Microcontrôleur | <1 MB | ~100 KB |

# 4.5 Synthèse décisionnelle

| Situation | Choix recommandé | Justification |
|---|---|---|
| Précision critique, serveur GPU | ResNet-101/152 | Meilleure accuracy |
| Production industrielle | ResNet-50 | Bon compromis |
| Application mobile | MobileNetV2/V3 | Léger et rapide |
| Prototype académique | VGG-16 | Simple à comprendre |
| IoT / Edge computing | MobileNetV3-Small | Ultra-léger |
| Détection d'objets | ResNet-50 + FPN | Backbone standard |

# Conclusion